

Beyond the GIL: Python's Journey to Free Threading

PyCon Israel, September 2025

bit.ly/nogil

Yonatan Bitton

@bityob

linktr.ee/bityob



whoami

- **Yonatan Bitton** — Core Team @ **Fortinet**, advanced cybersecurity protection
- Passionate about **Python**, enjoy solving tough **challenges** and understanding things **inside out**



Agenda

Agenda

- GIL Recap

Agenda

- GIL Recap
- Trials & Workarounds

Agenda

- GIL Recap
- Trials & Workarounds
- PEP 703

Agenda

- GIL Recap
- Trials & Workarounds
- PEP 703
- Short demo

Agenda

- GIL Recap
- Trials & Workarounds
- PEP 703
- Short demo
- Challenges & The Road Ahead

Agenda

- GIL Recap
- Trials & Workarounds
- PEP 703
- Short demo
- Challenges & The Road Ahead
- Summary

GIL Recap

- 1991: First public release — Python 0.9.1

- 1991: First public release — Python 0.9.1
- 1992: Threads + the GIL introduced

- 1991: First public release — Python 0.9.1
- 1992: Threads + the GIL introduced
 - GIL: only one thread runs Python at a time

- 1991: First public release — Python 0.9.1
- 1992: Threads + the GIL introduced
 - GIL: only one thread runs Python at a time
 - Simplified memory management and ensured thread safety

- 1991: First public release — Python 0.9.1
- 1992: Threads + the GIL introduced
 - GIL: only one thread runs Python at a time
 - Simplified memory management and ensured thread safety
 - Made sense in the **single-core era**

CPU 1

Thread 1

RUN (1)

RUN (1)

RUN (1)

Thread 2

RUN (1)

RUN (1)

RUN (1)

GIL

GIL

GIL

GIL

GIL

Trials & Workarounds

Why Not the GIL?

Why Not the GIL?

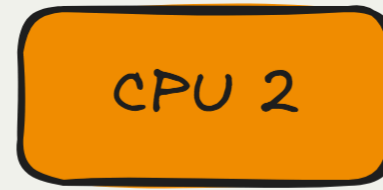
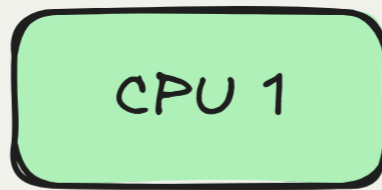
- Blocks true multi-core parallelism

Why Not the GIL?

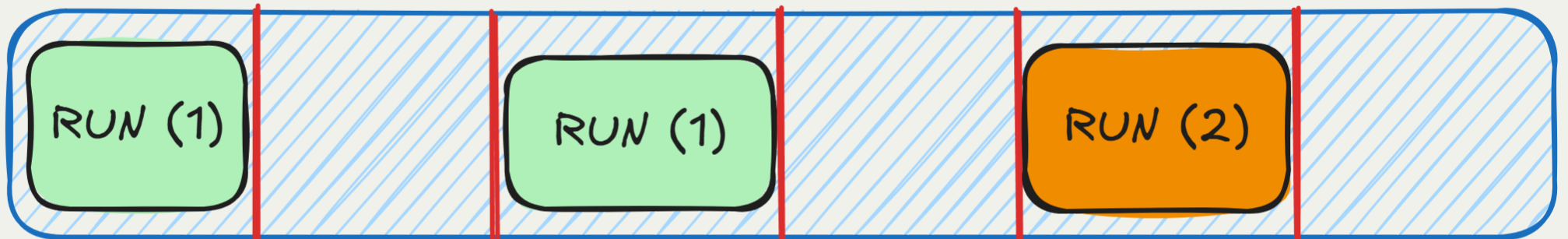
- Blocks true multi-core parallelism
- Bottleneck for CPU-bound code

Why Not the GIL?

- Blocks true multi-core parallelism
- Bottleneck for CPU-bound code
- Wastes modern multi-core CPUs



Thread 1



Thread 2



GIL

GIL

GIL

GIL

GIL

Why Not Just Remove It?

Why Not Just Remove It?

- Deeply embedded in memory management

Why Not Just Remove It?

- Deeply embedded in memory management
- Breaks interpreter assumptions

Why Not Just Remove It?

- Deeply embedded in memory management
- Breaks interpreter assumptions
- Race conditions everywhere

Why Not Just Remove It?

- Deeply embedded in memory management
- Breaks interpreter assumptions
- Race conditions everywhere

But that hasn't stopped attempts to remove it...

- 1996: Greg Stein's "free threading" patch

- 1996: Greg Stein's "free threading" patch
- 2007: python-safethread by Adam Olsen

- 1996: Greg Stein's "free threading" patch
- 2007: python-safethread by Adam Olsen
- 2016: Gilectomy (Larry Hastings)

The GIL survives.

What About Workarounds?

What About Workarounds?

- AsyncIO: concurrency, not parallelism

What About Workarounds?

- AsyncIO: concurrency, not parallelism
- C extensions: can release GIL — but risky

What About Workarounds?

- AsyncIO: concurrency, not parallelism
- C extensions: can release GIL — but risky
- Multiprocessing: works — but heavyweight

What About Workarounds?

- AsyncIO: concurrency, not parallelism
- C extensions: can release GIL — but risky
- Multiprocessing: works — but heavyweight

Developers want **real** threads

Still Searching for a Solution...

- 2021: nogil (Sam Gross @ Meta)

- 2021: nogil (Sam Gross @ Meta)
- 2023: PEP 703 published (Jan 2023)

PEP 703 – Making the Global Interpreter Lock Optional in CPython

Author: Sam Gross <colesbury at gmail.com>

Sponsor: Łukasz Langa <lukasz at python.org>

Discussions-To: [Discourse thread](#)

Status: [Accepted](#)

Type: [Standards Track](#)

Created: 09-Jan-2023

Python-Version: 3.13

Post-History: [09-Jan-2023](#), [04-May-2023](#)

Resolution: [24-Oct-2023](#)

- 2023: Meta supports and commits three engineer-years through 2025



A fast, free threading Python

■ Ideas



Carl Meyer carljm CPython core developer

Jul 2023



Guido van Rossum:



it would be great if Meta or another tech company could spare some engineers with established CPython internals experience to help the core dev team with this work.

We've had a chance to discuss this internally with the right people. Our team believes in the value that nogil will provide, and we are committed to working collaboratively to improve Python for everyone.

If PEP 703 is accepted, Meta can commit to support in the form of three engineer-years (from engineers experienced working in CPython internals) between the acceptance of PEP 703 and the end of 2025, to collaborate with the core dev team on landing the PEP 703 implementation smoothly in CPython and on ongoing improvements to the compatibility and performance of nogil CPython.

- 2023: PEP 703 **accepted** with clear proviso (Oct 2023)

Note

The Steering Council accepts PEP 703, but with clear proviso: that the rollout be gradual and break as little as possible, and that we can roll back any changes that turn out to be too disruptive – which includes potentially rolling back all of PEP 703 entirely if necessary (however unlikely or undesirable we expect that to be).

- 2024: Python 3.13 with **experimental** free-threaded build
 - Opt-in via `--disable-gil` compile flag

The free-threaded mode is experimental and work is ongoing to improve it: expect some bugs and a substantial single-threaded performance hit.

- 2025: PEP 779: Criteria for supported status

- 2025: PEP 779: Criteria for supported status
- 2025: Python 3.14.0rc2 released in August 2025

- 2025: PEP 779: Criteria for supported status
- 2025: Python 3.14.0rc2 released in August 2025
 - Not **experimental**, not yet the default build

- 2025: PEP 779: Criteria for supported status
- 2025: Python 3.14.0rc2 released in August 2025
 - Not **experimental**, not yet the default build
- 2030~: Python default build?

How to use?

```
$ uv python install 3.14t  
Installed Python 3.14.0b4 in 12.54s  
+ cpython-3.14.0b4+freethreaded-linux-x86_64-gnu (python3.14t)
```

```
1 # demo-freethreaded-python.py
2 import time
3 from concurrent.futures import ThreadPoolExecutor
4
5 N = 50_000_000
6 THREADS = 10
7
8 def work(n):
9     for _ in range(n):
10         pass
11
12 start = time.time()
13
14 with ThreadPoolExecutor(max_workers=THREADS) as pool:
15     list(pool.map(work, [N] * THREADS))
16
```

```
3 from concurrent.futures import ThreadPoolExecutor
4
5 N = 50_000_000
6 THREADS = 10
7
8 def work(n):
9     for _ in range(n):
10         pass
11
12 start = time.time()
13
14 with ThreadPoolExecutor(max_workers=THREADS) as pool:
15     list(pool.map(work, [N] * THREADS))
16
17 print(f"Done in {time.time() - start:.2f} sec")
```

```
$ uv run --python=3.14 python demo-freethreaded-python.py  
Done in 9.39 sec
```

```
$ uv run --python=3.14t python demo-freethreaded-python.py  
Done in 0.99 sec
```


Removing the GIL: What Changed

Reference Counting

Reference Counting

- Each object tracks how many references point to it

Reference Counting

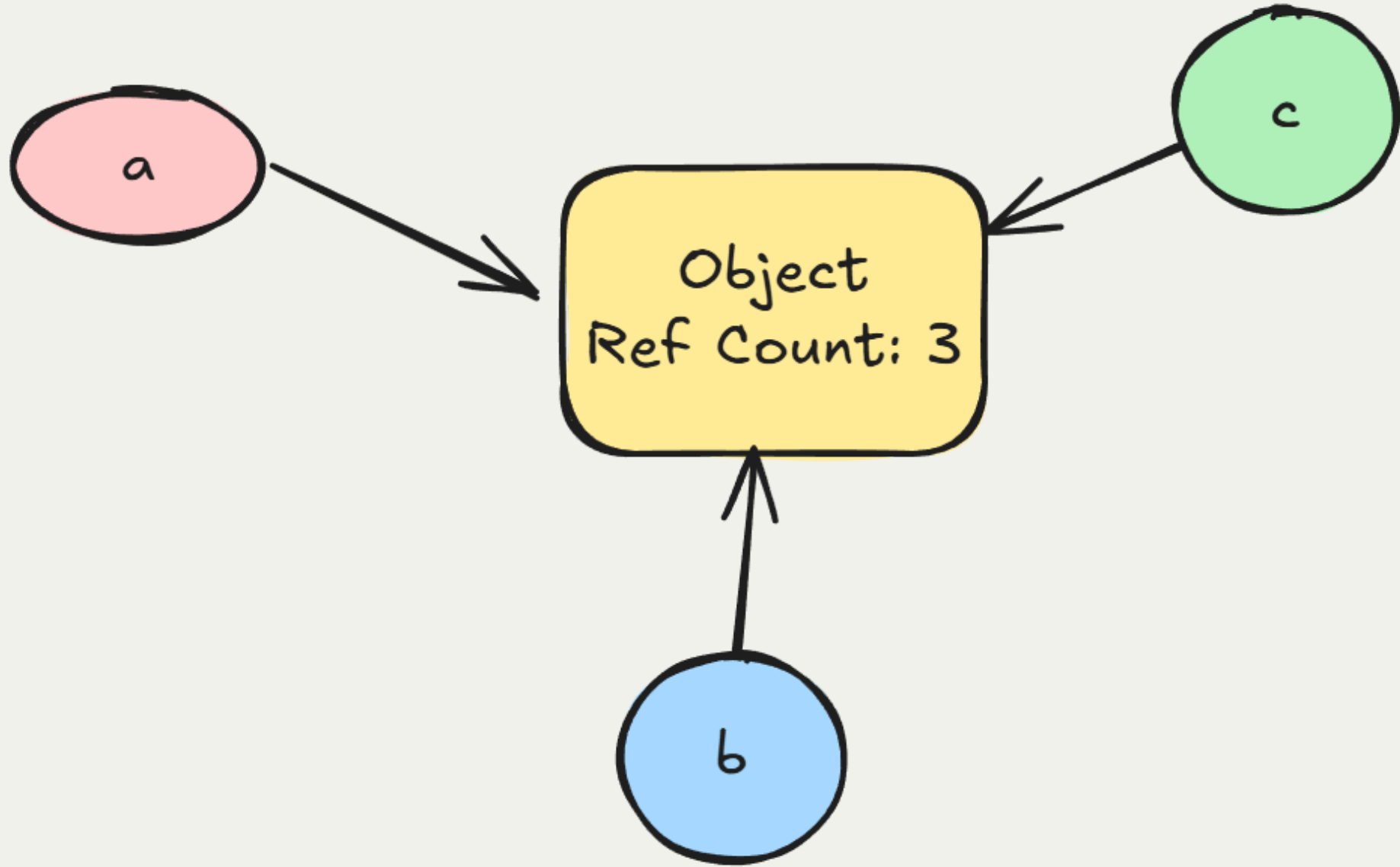
- Each object tracks how many references point to it
- Used in Python for predictable, immediate cleanup

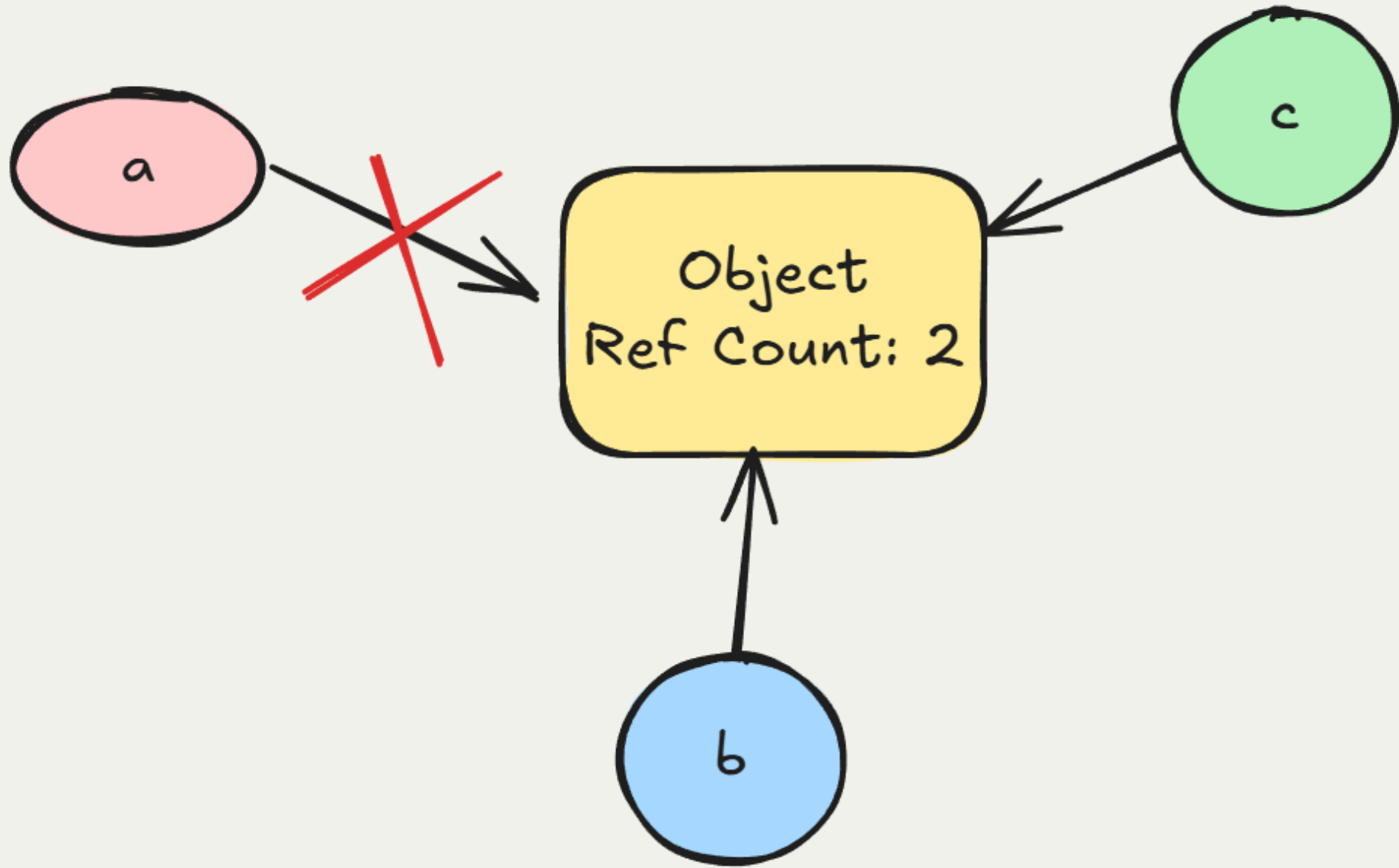
Reference Counting

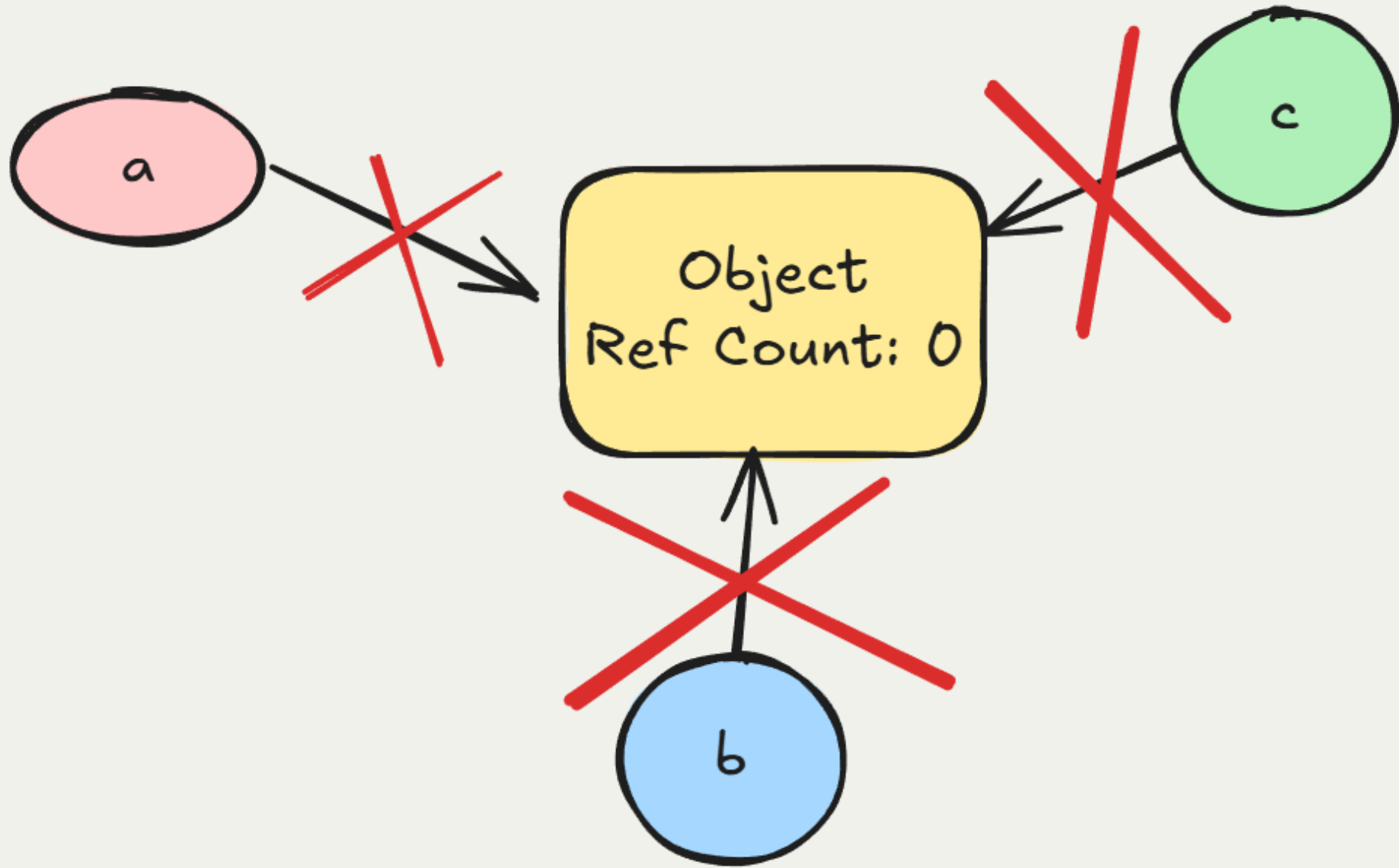
- Each object tracks how many references point to it
- Used in Python for predictable, immediate cleanup
- Count goes up on new refs, down on delete/reassign

Reference Counting

- Each object tracks how many references point to it
- Used in Python for predictable, immediate cleanup
- Count goes up on new refs, down on delete/reassign
- When it hits zero → object is freed







Why Reference Counting Is a Problem

- GIL protects reference count updates today
- Without GIL → race conditions

Biased Reference Counting

Biased Reference Counting

- Objects are **biased** to the creating thread

Biased Reference Counting

- Objects are **biased** to the creating thread
- Owning thread updates refcount **without atomic ops**

Biased Reference Counting

- Objects are **biased** to the creating thread
- Owning thread updates refcount **without atomic ops**
- Other threads must use **atomic operations** → slower

Biased Reference Counting

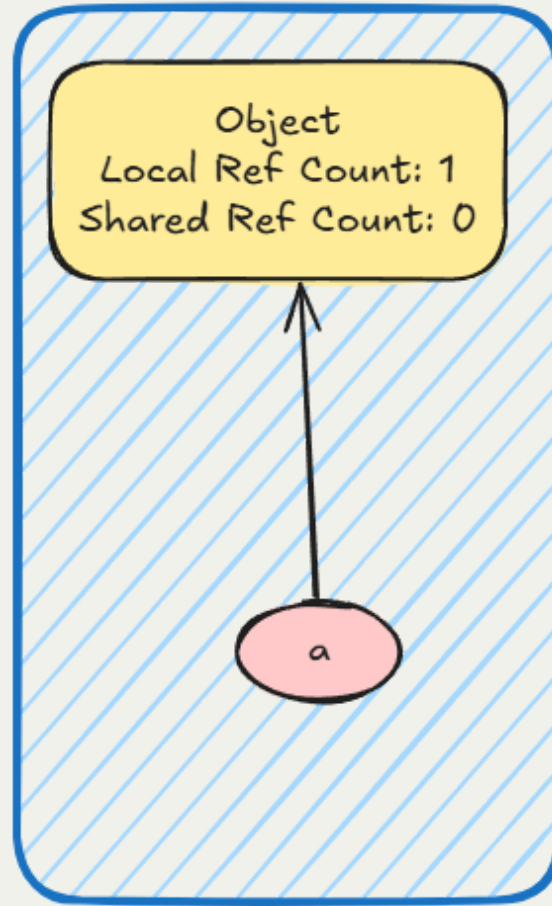
- Objects are **biased** to the creating thread
- Owning thread updates refcount **without atomic ops**
- Other threads must use **atomic operations** → slower
- Efficient because **most objects stay in one thread**

```
1 struct _object {
2     _PyObject_HEAD_EXTRA
3     uintptr_t ob_tid;           // owning thread id (4-8 bytes)
4     uint16_t __padding;        // reserved for future use (2 bytes)
5     PyMutex ob_mutex;          // per-object mutex (1 byte)
6     uint8_t ob_gc_bits;        // GC fields (1 byte)
7     uint32_t ob_ref_local;      // local reference count (4 bytes)
8     Py_ssize_t ob_ref_shared;  // shared reference count and state bits (4-8 bytes)
9     PyObject *ob_type;
10 };
```

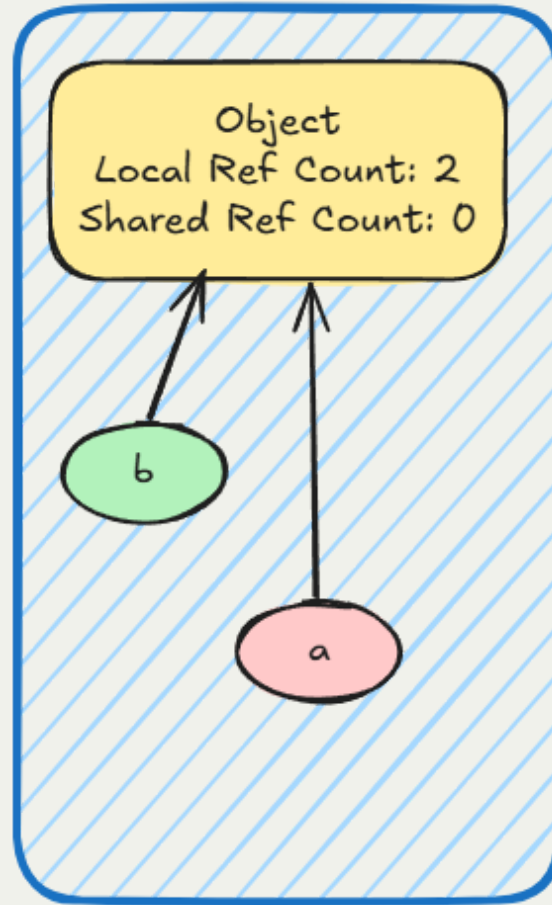


```
1 struct _object {
2     _PyObject_HEAD_EXTRA
3     uintptr_t ob_tid;           // owning thread id (4-8 bytes)
4     uint16_t __padding;        // reserved for future use (2 bytes)
5     PyMutex ob_mutex;          // per-object mutex (1 byte)
6     uint8_t ob_gc_bits;        // GC fields (1 byte)
7     uint32_t ob_ref_local;      // local reference count (4 bytes)
8     Py_ssize_t ob_ref_shared;  // shared reference count and state bits (4-8 bytes)
9     PyObject *ob_type;
10 };
```

Thread 1

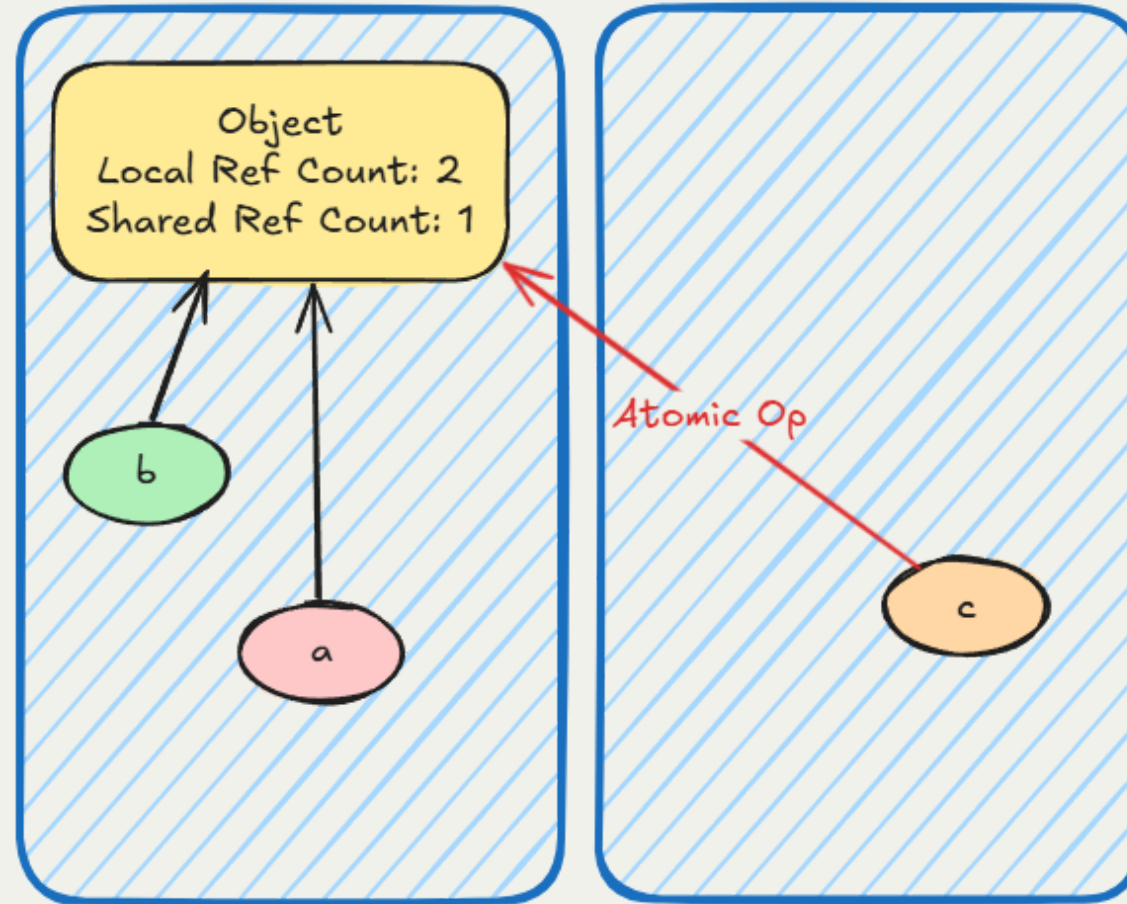


Thread 1



Thread 1

Thread 2



Immortal Objects

Immortal Objects

- Common objects are never deallocated

Immortal Objects

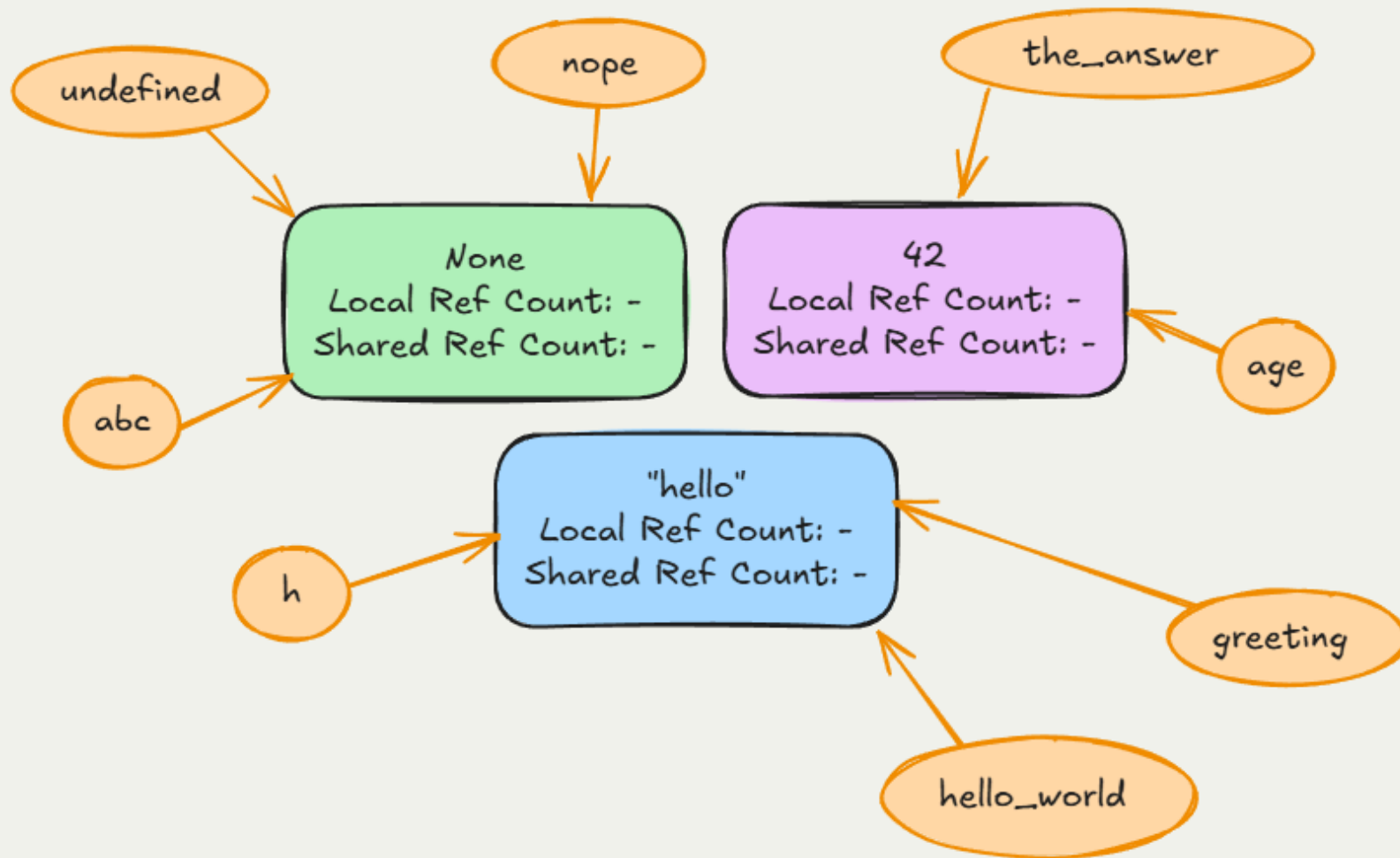
- Common objects are never deallocated
 - True, False, None, interned strings, small integers, etc.

Immortal Objects

- Common objects are never deallocated
 - True, False, None, interned strings, small integers, etc.
- Skip reference count updates **at all**

Immortal Objects

- Common objects are never deallocated
 - True, False, None, interned strings, small integers, etc.
- Skip reference count updates **at all**
- Reduce thread contention



Deferred Reference Counting

Deferred Reference Counting

- Skip reference count changes for hot objects

Deferred Reference Counting

- Skip reference count changes for hot objects
 - Code, functions, modules, etc.

Deferred Reference Counting

- Skip reference count changes for hot objects
 - Code, functions, modules, etc.
 - Reference counting on every access is inefficient

Deferred Reference Counting

- Skip reference count changes for hot objects
 - Code, functions, modules, etc.
 - Reference counting on every access is inefficient
- GC handles cleanup later

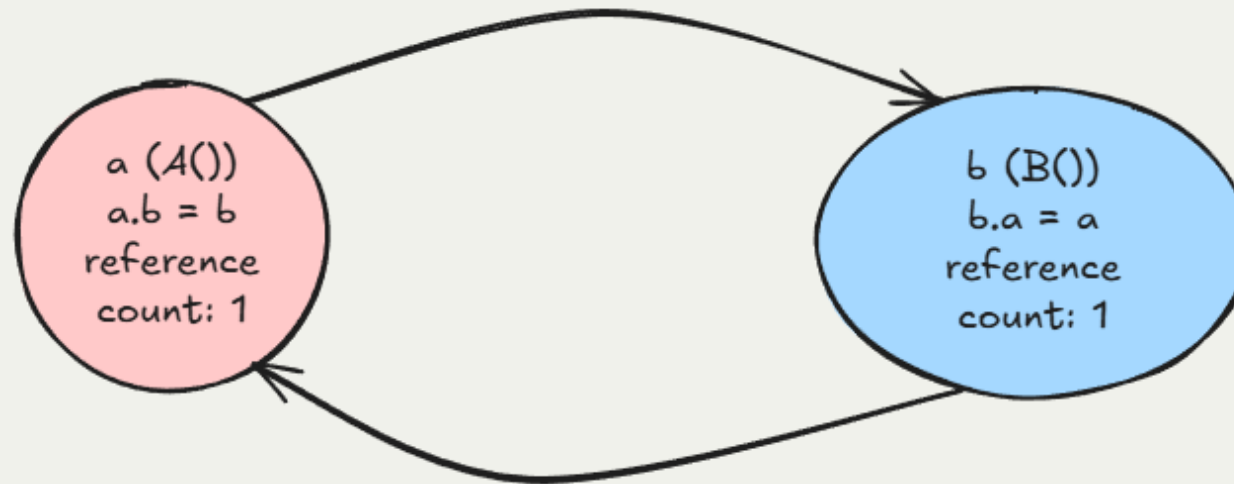
Garbage Collection (GC)

Garbage Collection (GC)

- Detects and collects **cyclic references** that reference counting misses

Garbage Collection (GC)

- Detects and collects **cyclic references** that reference counting misses

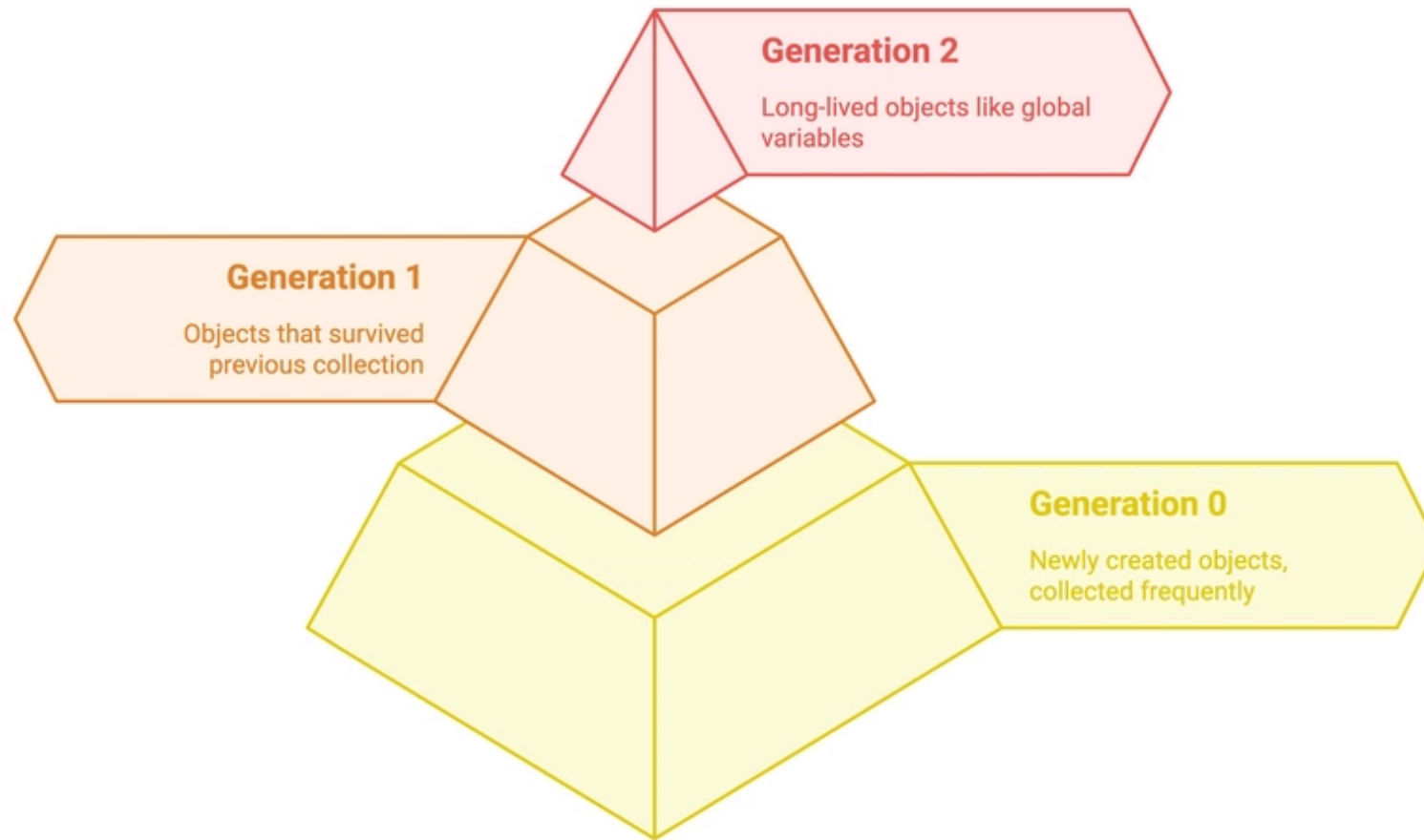


Garbage Collection (GC)

Garbage Collection (GC)

- Uses **generational GC**: objects are grouped by age; older ones are scanned less often

Object Lifecycle in Garbage Collection



GC without GIL

GC without GIL

- Can't rely on the GIL to **synchronize memory access** — GC must now be **explicitly thread-safe**

GC without GIL

- Can't rely on the GIL to **synchronize memory access** — GC must now be **explicitly thread-safe**
- “Stop-the-world” GC needed

GC without GIL

- Can't rely on the GIL to **synchronize memory access** — GC must now be **explicitly thread-safe**
- “Stop-the-world” GC needed
 - All threads pause during collection to avoid race conditions

GC without GIL

GC without GIL

- Single-generation GC

GC without GIL

- Single-generation GC
 - Using a **single generation** avoids frequent pauses

GC without GIL

- Single-generation GC
 - Using a **single generation** avoids frequent pauses
 - **Generational GC is less critical** in Python — young objects are usually cleaned up early via **reference counting**

GC without GIL

- Single-generation GC
 - Using a **single generation** avoids frequent pauses
 - **Generational GC is less critical** in Python — young objects are usually cleaned up early via **reference counting**
- Integration with deferred and biased reference counting

Per-object locks

Per-object locks

- Global locks like the **GIL** add overhead but **no deadlocks** if only one lock exists

Per-object locks

- Global locks like the **GIL** add overhead but **no deadlocks** if only one lock exists
- Using **multiple locks** can cause **deadlocks** (e.g., double locks)

Per-object locks

- Global locks like the **GIL** add overhead but **no deadlocks** if only one lock exists
- Using **multiple locks** can cause **deadlocks** (e.g., double locks)
- **Per-object locks**: each object has its own lock
 - Acquired only when accessing that object
 - Released if thread needs another lock or pauses

```
1 struct _object {
2     _PyObject_HEAD_EXTRA
3     uintptr_t ob_tid;           // owning thread id (4-8 bytes)
4     uint16_t __padding;        // reserved for future use (2 bytes)
5     PyMutex ob_mutex;          // per-object mutex (1 byte)
6     uint8_t ob_gc_bits;        // GC fields (1 byte)
7     uint32_t ob_ref_local;      // local reference count (4 bytes)
8     Py_ssize_t ob_ref_shared;  // shared reference count and state bits (4-8 bytes)
9     PyObject *ob_type;
10 };
```

```
1 struct _object {
2     _PyObject_HEAD_EXTRA
3     uintptr_t ob_tid;           // owning thread id (4-8 bytes)
4     uint16_t __padding;        // reserved for future use (2 bytes)
5     PyMutex ob_mutex;          // per-object mutex (1 byte)
6     uint8_t ob_gc_bits;        // GC fields (1 byte)
7     uint32_t ob_ref_local;      // local reference count (4 bytes)
8     Py_ssize_t ob_ref_shared;  // shared reference count and state bits (4-8 bytes)
9     PyObject *ob_type;
10 };
```

Containers Thread-Safety

Containers Thread-Safety

- `list`, `dict`, `set` use **per-object locks**

Containers Thread-Safety

- `list`, `dict`, `set` use **per-object locks**
- Modifying operations **acquire the lock**

Containers Thread-Safety

- `list`, `dict`, `set` use **per-object locks**
- Modifying operations **acquire the lock**
- Some read operations (e.g., `len()`) **skip the lock**

Containers Thread-Safety

- `list`, `dict`, `set` use **per-object locks**
- Modifying operations **acquire the lock**
- Some read operations (e.g., `len()`) **skip the lock**
- **Optimistic access**: try **lock-free** reads first; fallback to per-object lock if needed

Optimistic access

Optimistic access

- Read list/dict elements **without locking** when possible

Optimistic access

- Read list/dict elements **without locking** when possible
- Improves **multi-threaded scalability** and **single-threaded performance**

Optimistic access

- Read list/dict elements **without locking** when possible
- Improves **multi-threaded scalability** and **single-threaded performance**
- Retrieve item and **increment refcount only if > 0**

Optimistic access

- Read list/dict elements **without locking** when possible
- Improves **multi-threaded scalability** and **single-threaded performance**
- Retrieve item and **increment refcount only if > 0**
- Perform **extra checks** to ensure the object wasn't freed or replaced

```
1 // Attempt to read the item without a lock.
2 // This is the optimistic, lock-free path
3 PyObject** ob_item = atomic_load(&a->ob_item);
4 PyObject* item = atomic_load(&ob_item[i]);
5
6 // This is the optimistic check. If the item is NULL, or if _Py_TRY_INCREF
7 // fails (e.g., because the object's reference count is zero), we fall back to
8 // the safe, locked path.
9 if (!item || !_Py_TRY_INCREF(item)) {
10     goto retry;
11 }
12
13 // More checks...
14 // If all checks pass, we have a valid, thread-safe reference.
15 return item;
```

```
1 // Attempt to read the item without a lock.
2 // This is the optimistic, lock-free path
3 PyObject** ob_item = atomic_load(&a->ob_item);
4 PyObject* item = atomic_load(&ob_item[i]);
5
6 // This is the optimistic check. If the item is NULL, or if _Py_TRY_INCREF
7 // fails (e.g., because the object's reference count is zero), we fall back to
8 // the safe, locked path.
9 if (!item || !_Py_TRY_INCREF(item)) {
10     goto retry;
11 }
12
13 // More checks...
14 // If all checks pass, we have a valid, thread-safe reference.
15 return item;
```



```
1 // Attempt to read the item without a lock.
2 // This is the optimistic, lock-free path
3 PyObject** ob_item = atomic_load(&a->ob_item);
4 PyObject* item = atomic_load(&ob_item[i]);
5
6 // This is the optimistic check. If the item is NULL, or if _Py_TRY_INCREF
7 // fails (e.g., because the object's reference count is zero), we fall back to
8 // the safe, locked path.
9 if (!item || !_Py_TRY_INCREF(item)) {
10     goto retry;
11 }
12
13 // More checks...
14 // If all checks pass, we have a valid, thread-safe reference.
15 return item;
```

```
// The 'retry' label handles the pessimistic, lock-based path.
retry:
    PyObject* retried_item;

    // Use a critical section to ensure exclusive access.
    Py_BEGIN_CRITICAL_SECTION(a->ob_mutex);

    retried_item = a->ob_item[i]; // Read the item from the list.
    Py_INCREF(retried_item); // Increment the reference count.

    Py_END_CRITICAL_SECTION(a->ob_mutex);

    return retried_item;
```

Open Issues

Open Issues

- Still behind build flag

Open Issues

- Still behind build flag
- Library & package adoption

Open Issues

- Still behind build flag
- Library & package adoption
- Maintaining two runtimes

Open Issues

- Still behind build flag
- Library & package adoption
- Maintaining two runtimes
- Debugging & profiling gaps

Open Issues

- Still behind build flag
- Library & package adoption
- Maintaining two runtimes
- Debugging & profiling gaps
- Single-threaded performance overhead

Open Issues

- Still behind build flag
- Library & package adoption
- Maintaining two runtimes
- Debugging & profiling gaps
- Single-threaded performance overhead
- When is it stable?

Takeaways

Takeaways

- The **GIL** is a bottleneck for multi-core applications

Takeaways

- The **GIL** is a bottleneck for multi-core applications
- To enable free threading while maintaining performance, **PEP 703** revamps Python's internals with thread-safe memory management

Takeaways

- The **GIL** is a bottleneck for multi-core applications
- To enable free threading while maintaining performance, **PEP 703** revamps Python's internals with thread-safe memory management
- The new **experimental build** unlocks massive speed-ups for multi-threaded programs

Takeaways

- The **GIL** is a bottleneck for multi-core applications
 - To enable free threading while maintaining performance, **PEP 703** revamps Python's internals with thread-safe memory management
 - The new **experimental build** unlocks massive speed-ups for multi-threaded programs
 - This is a **monumental step** toward a parallel future for the Python you know
-

Any Questions?

bit.ly/nogil

Yonatan Bitton
@bityob
linktr.ee/bityob



Sources

- PEP 703 – Making the Global Interpreter Lock Optional in CPython ([peps.python.org](https://peps.python.org/pep-0703/))
- PEP 779 – Criteria for supported status for free-threaded Python ([peps.python.org](https://peps.python.org/pep-0779/))
- Python Free-Threading Guide (py-free-threading.github.io)

Learn More

- Multithreaded Python without the GIL - presented by Sam Gross (youtube.com)
 - High-Performance Python: Faster Type Checking and Free Threaded Execution (youtube.com)
 - Python Docs How To - Python experimental support for free (docs.python.org)
 - Understanding the Python GIL - by David Beazley (dabeaz.com)
 - Biased Reference Counting: Minimizing Atomic Operations in Garbage Collection (dl.acm.org)
-